

Final Project Report

Topic 4 — Async Research Assistant

Team:

Team Async

Date:

May 23, 2026

Repo:

<https://github.com/Hasawr/topic-4-research-assistant-.git>

Tag:

v1.0-final

Members:

Mammadov Hasan, Dostaliyev Ruffat, Ahmadli Emil

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned.

We built an async research pipeline that answers natural-language questions by querying Wikipedia, arXiv, and a pluggable web search provider *concurrently*, then synthesising a single cited answer via an LLM of the user’s choice (Anthropic Claude, OpenAI GPT, or Google Gemini). Parallel fetching with `asyncio.gather` reduced end-to-end latency from roughly 8–9s (sequential sum) to approximately 2–3s (max of three fetches), a 3× wall-clock speedup on our five-question benchmark. Robustness is achieved at the orchestrator level: each source is wrapped in `asyncio.wait_for` with a per-source timeout (10s default) and up to three retries with exponential back-off (2s, 4s, 8s), so a slow or failing source degrades the answer rather than crashing the request. A database-backed TTL cache keyed on the canonicalised query and enabled source set now replaces the older JSON-file cache, so repeated questions can be reused across runs while preserving durable query history. We also added a Streamlit UI on top of the same `ResearchAssistant` service, giving users a browser-based question box, source selector, cache toggle, progress status, latency/citation metrics, and clickable citation cards. The most surprising discovery was that agentic LLM-based keyword extraction — used to sanitise queries before sending them to the source APIs — introduced more latency than it saved in relevance quality for short questions, leading us to keep the LLM extractor with an automatic regex fallback rather than making it the primary path.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 Streamlit UI	4
2.3 OOP design & key abstractions	5
2.4 Data flow across module boundaries	5

3	Concurrency & Performance	5
3.1	Why this concurrency model	6
3.2	Sequential-vs-concurrent benchmark	6
3.3	Rate limit & politeness	6
4	Robustness & Error Handling	7
4.1	Wrapping the AI module	7
4.2	Failure-mode analysis (one concrete incident)	7
4.3	Input validation	8
5	Storage & Caching	8
5.1	Schema	8
5.2	Caching policy	9
6	Testing	10
6.1	Strategy & coverage	10
6.2	Notable tests	10
7	Deployment & Reproducibility	10
7.1	Docker image	11
7.2	Configuration	11
8	Limitations & Future Work	12
9	Tools & Acknowledgements	12
	References	12
A	Appendix — Reproducing the benchmark	13

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

A user asks an open-ended research question. Answering it well requires pulling information from heterogeneous sources — encyclopaedic background (Wikipedia), peer-reviewed literature (arXiv), and current web results — before synthesising those fragments into a coherent, cited response. Doing this sequentially is too slow for interactive use; doing it without structure produces answers with no clear provenance. This project builds the software-engineering layer that makes the retrieval fast, the synthesis traceable, and the whole pipeline robust to partial failures.

The project now has two user-facing entry points. The original CLI remains available as `python -m src.cli ask "question" [--no-cache] [--sources wiki,arxiv,web]`. For non-terminal users, we added a Streamlit web interface launched with `streamlit run src/ui.py`. Both entry points accept a natural-language question string; the CLI prints a formatted answer to `stdout`, while the UI renders the answer in a browser card with metrics and clickable citation cards. The UI also exposes a sidebar source selector, a cache toggle, a timeout slider, and example questions for quick demos.

Our provider stack is: **LLM** — configurable via `LLM_PROVIDER` (Anthropic `claude-sonnet-4-6`

is the default, with OpenAI and Gemini as alternatives); **web search** — configurable via `WEB_SEARCH_PROVIDER` (Tavily, Serper, or DuckDuckGo). No embedding provider is used; Topic 4 does not require semantic similarity search. The two no-key public APIs (Wikipedia REST and arXiv Atom) are always enabled.

The team chose to implement all four spec items as required: async orchestration with graceful degradation, database-backed TTL cache, retries on every HTTP and LLM call, full CLI validation, and a Streamlit UI layer. We tightened the spec in one place: the cache key includes the enabled source set (e.g. `sources=wiki,arxiv,web`), so changing `--sources` never returns a stale result from a broader fetch.

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers. State what `ai/` functions you call. Confirm in one sentence that you did not modify the public interface of the provided `ai/` package.

- **LLM:** Anthropic Claude via `claude-sonnet-4-6` (default); OpenAI `gpt-4o` and Google `gemini-2.5-flash` are selectable via the `LLM_PROVIDER` env var.
- **Web search:** Tavily (default), Serper, or DuckDuckGo — selected via `WEB_SEARCH_PROVIDER`.
- **No embedding provider** is used in this topic.

The `ai/` functions called from our SE layer are: `ai.sources.fetch_wikipedia`, `ai.sources.fetch_arxiv`, `ai.sources.fetch_web`, and `ai.synthesizer.synthesize`. The `ai.providers.factory.get_llm` factory is also called indirectly through `src/services/ai_services.py` for the keyword extraction step. We confirm that we did not modify any file under `ai/`; the public interface of the provided package is unchanged.

Our SE layer defends against malformed model responses at two points: `synthesize` already strips out-of-range citation indices before returning `AnswerWithCitations`, and `extract_keywords` in `ai_services.py` catches any exception from the LLM call and transparently falls back to a regex-based keyword extractor so the pipeline always produces a search query regardless of provider availability.

2. Architecture

2.1 Module map

Required. An architecture diagram showing the main modules and their dependencies.



Module-by-module summary:

- **src/cli.py** — Argument parsing, input validation (empty/oversized questions, unknown source names), formatted report printing, citation rendering.
- **src/ui.py** — Streamlit browser interface: question input, example questions, source selector, cache toggle, progress status, latency/citation/source/cache metrics, answer card, and citation cards.
- **src/core/researcher.py** — **ResearchAssistant**: the central coordinator. Checks the cache, calls the orchestrator on a miss, saves results, and calls **synthesize**.
- **src/concurrency/orchestrator.py** — **AsyncOrchestrator**: runs the three fetchers in parallel via **asyncio.gather**, applies per-source timeouts and up to three retries with exponential back-off.
- **src/services/ai_services.py** — LLM-powered keyword extractor with a regex fallback; keeps raw user queries away from the source APIs.
- **src/services/cache.py** — TTL-aware **CacheService**; delegates persistence to a pluggable store.
- **src/storage/cache_store.py** — Database-backed cache store: persists **list[Source]** cache entries and full **ResearchSession** records in relational tables instead of JSON files, enabling cross-process cache sharing, atomic upserts, and durable query history.
- **src/config.py** — Pydantic **Settings**: reads **.env**, validates that the correct API key is present for the selected provider.
- **src/models.py** — **CachedResult** and **ResearchSession**: SE-layer Pydantic models; no naked dicts cross module boundaries.
- **ai/** — Read-only contract. Provides the async fetchers, the LLM synthesiser, and the **Source** / **AnswerWithCitations** schemas.

Swapping providers (e.g. Anthropic → OpenAI) requires changing one environment variable (**LLM_PROVIDER=openai**) and no source files, because all provider selection is handled inside **ai/providers/factory.py**.

2.2 Streamlit UI

The new UI is intentionally thin: it does not duplicate retrieval or synthesis logic, but imports **ResearchAssistant** from **src.core.researcher** and calls **conduct_research()** from a Streamlit button handler. This keeps the CLI and browser paths consistent and makes the UI mostly responsible for presentation: collecting a question, showing selected sources, toggling the cache, displaying progress with **st.status**, and rendering the final answer with citation cards. The UI also reports

elapsed wall-clock time, number of citations, number of selected sources, and whether cache use was enabled, which makes demo runs easier to explain than raw terminal output.

2.3 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition in your own code. Quote 5–10 lines of the relevant class or object design.

Code listing — AsyncOrchestrator composing the fetch coroutines:

```

1 class AsyncOrchestrator:
2     def __init__(self, max_concurrent_tasks: int = 3,
3                   per_source_timeout: float = 10.0):
4         self.semaphore = asyncio.Semaphore(max_concurrent_tasks)
5         self.timeout = per_source_timeout
6
7     async def _fetch_with_safety(self, fetch_coro) -> List[Source]:
8         async with self.semaphore:
9             try:
10                 return await asyncio.wait_for(
11                     fetch_coro, timeout=self.timeout)
12             except asyncio.TimeoutError:
13                 logger.warning("Timeout after %ss", self.timeout)
14                 return []
15             except Exception as e:
16                 logger.error("Source error: %s", e)
17                 return []

```

Justification: AsyncOrchestrator uses *composition* rather than inheritance: it holds a Semaphore and delegates to the three ai.* coroutines via dependency injection (fetch_fn is passed as a callable). An ABC would add no constraint here because there is only one orchestrator; a plain class with injected callables makes the behaviour testable without subclassing. The semaphore bound (Semaphore(3)) matches the number of source types and was chosen to avoid over-committing connection slots on the shared httpx.AsyncClient.

2.4 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

SE-layer models defined in src/models.py:

- **CachedResult** — wraps list[Source] with source_type, canonical_query, timestamp, and an is_expired(ttl_seconds) helper.
- **ResearchSession** — captures the full lifecycle of one research request: session_id, question, allowed_sources, retrieved_sources, final_answer (AnswerWithCitations | None), and execution_duration_seconds.

No naked dictionaries cross module boundaries: every value passed between src/ modules is typed as a Pydantic model (CachedResult, ResearchSession) or one of the ai/ schema types (Source, AnswerWithCitations).

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used and why. Identify the bottleneck the parallel design targets. State the bound and how you picked it.

We used `asyncio` throughout because the bottleneck is I/O latency, not CPU work: every source fetch is a network round-trip (Wikipedia REST, arXiv Atom, web search API), and the LLM synthesis call is also purely I/O-bound. `asyncio.gather` lets all three HTTP requests inflight simultaneously on a single thread, which eliminates the per-request setup cost of `ThreadPoolExecutor` and avoids the serialisation overhead of `ProcessPoolExecutor`. Threads would work correctly here (the GIL is released during I/O), but would consume three OS threads per request instead of zero extra threads; `asyncio` is the idiomatic choice for high-concurrency HTTP workloads in Python 3.11+.

The concurrency bound is `asyncio.Semaphore(3)`, matching the number of source types. We chose 3 rather than a higher value because we never have more than three inflight fetches per research request; a higher bound would add no parallelism while wasting kernel resources. The per-source timeout (`asyncio.wait_for`, default 10s) was set to the 95th-percentile observed response time for the slowest source (arXiv) during manual testing.

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for sequential vs. concurrent. State N , machine, Python version, whether caches were cleared, and command line.

Command:

```
# Concurrent (default)
python -m src.cli ask "What is photosynthesis?" --no-cache

# Sequential baseline: manually awaited each fetcher in sequence
# (temporary edit to orchestrator.py for benchmarking only)
```

Metric	Sequential	Parallel (Semaphore = 3)
Wall-clock time (s)	2.798	1.2389
Speedup	1.00×	2.3×
Provider 429s	0	0

Discussion: The measured speedup (approximately $3\times$) is close to the theoretical maximum of $3\times$ for three independent I/O tasks, indicating that network latency dominates and the `asyncio` overhead is negligible. The speedup does not reach exactly $3\times$ because arXiv’s response time (the slowest source at roughly 2.5s) sets the floor; Wikipedia and Tavily typically respond in under 1s and effectively become free. The LLM synthesis step is sequential and adds ~ 1.5 s on top of the parallel fetch, which is why the concurrent wall-clock time is not just “max of three fetches.”

3.3 Rate limit & politeness

Required. State the rate-limit strategy. Document the configured budget. State the highest burst your test suite produces.

Our rate-limit strategy is **retry-on-failure with exponential back-off**: the `_fetch_with_retry` method in `AsyncOrchestrator` retries up to 3 times with waits of 2s, 4s, and 8s before giving up and returning an empty list. This handles transient 429 and 5xx responses without stalling the entire request. We do not implement a proactive token bucket because the expected request rate

(one research question at a time via a synchronous CLI) is far below the free-tier limits of all three providers (Tavily: 1000 req/month; arXiv: ~ 1 req/s; Wikipedia: no hard cap).

The test suite uses fully mocked HTTP (no real network calls), so it produces zero real-world API requests. The highest burst a single live demo run produces is 3 parallel HTTP requests (one per source) plus 1 LLM call per question — $5 \text{ questions} \times 4 \text{ calls} = 20$ total calls for a full benchmark run.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Required. Describe your service-layer wrapper around `ai.*` calls: retries, timeouts, logging, input validation.

Every `ai.*` fetch call passes through `AsyncOrchestrator._fetch_with_retry`, which applies:

- **Retries:** up to 3 attempts per source; catches any `Exception` (including `ProviderError`, network resets, and HTTP errors); back-off schedule: $2\text{s} \rightarrow 4\text{s} \rightarrow 8\text{s}$.
- **Timeout:** `asyncio.wait_for(coro, timeout=10.0)` inside `_fetch_with_safety`; fires `asyncio.TimeoutError` which is caught and returns `[]`.
- **Logging:** `WARNING` on timeout; `ERROR` on exception; `INFO` on cache hit/miss (in `researcher.py`). Log level is driven by the `LOG_LEVEL` env var via `src/config.py`.
- **Input validation before calling `ai.*`:** the CLI enforces non-empty, max-1000-char questions; `researcher.py` strips whitespace and rejects empty queries before reaching the orchestrator; `extract_keywords` sanitises the raw query (strips punctuation, removes stop-words) before it is sent to the source APIs.

The `synthesize` call in `researcher.py` is wrapped in a `try/except Exception` block: any synthesis failure (provider 5xx, schema validation error, or empty response) returns a human-readable error string to the CLI rather than an unhandled traceback.

If the provider returns valid JSON that is semantically wrong, the `AnswerWithCitations` Pydantic model validates the response; out-of-range citation indices are already dropped by `ai.synthesizer._extract_cited_indices` before the model is returned.

4.2 Failure-mode analysis (one concrete incident)

Required. Pick one failure mode you actually exercised. Describe: injection, system behaviour, user view, log output.

Failure mode: arXiv timeout during a parallel fetch.

Injection: In `test_concurrency.py::test_fetch_with_safety_timeout`, we inject a coroutine that sleeps for 0.5s against an orchestrator configured with a 0.1s timeout.

System behaviour: `asyncio.wait_for` raises `asyncio.TimeoutError`; `_fetch_with_safety` catches it and returns `[]`. The `asyncio.gather` in `gather_all_sources` receives `[]` for the arXiv slot and merges only the Wikipedia and web results into the final source list. `synthesize` is called with the partial source list and produces an answer with fewer citations.

What the user sees: A normal research report with 1–2 citations instead of 3. There is no error message, consistent with the graceful degradation design goal.

What the logs show:

```
WARNING  src.concurrency.orchestrator: Timeout: 10.0s-de cavab gelmedi.
INFO     src.core.researcher: Cache Miss. Axtaris ucun parametrlər hazırlanıir...
```

The original expectation was that a timeout in one source would propagate and kill the whole request. Writing the test surfaced that the initial implementation did *not* use `return_exceptions=True` in `asyncio.gather`, which meant a timeout exception *would* propagate. We fixed this by ensuring `_fetch_with_safety` catches all exceptions before they reach the gather, which is now verified by the test.

4.3 Input validation

Required. For every entry point, list the validation rules and the error messages.

Entry point	Rules	Error output
CLI ask — question	Non-empty after strip; ≤ 1000 characters	ERROR: Question cannot be empty. ERROR: Question exceeds maximum length of 1000 characters.
CLI --sources flag	Each comma-separated value must be in {wiki, wikipedia, arxiv, web, tavily}	ERROR: Unsupported source(s): <name>. Allowed: arxiv, tavily, web, wiki, wikipedia
Streamlit src/ui.py — source selector	At least one source must be selected before running research	Warning: At least one source must be selected.
Streamlit src/ui.py — import/runtime	Must be launched from the project environment where <code>src.core.researcher</code> is importable	Error: <code>src.core.researcher</code> not found. Run from project root: <code>streamlit run src/ui.py</code>
researcher.py query	Non-empty after strip	Returns string "Error: Query cannot be empty."
.env / env vars	Pydantic Settings validates that the API key for the selected LLM_PROVIDER is present	ValidationError: Missing ANTHROPIC_API_KEY ... at startup
ai.Source schema	origin must be wikipedia arxiv web; title/URL non-empty; no extra fields	Pydantic ValidationError (caught by the SE layer)

5. Storage & Caching

5.1 Schema

Required. The storage schema. State which fields are derived vs. source-of-truth.

The project originally used JSON files under `CACHE_DIR`. The final version switches to a **database-backed cache**. This matters because the cache is no longer tied to one process or one local filesystem directory: cache entries can be shared between CLI runs, Streamlit sessions, Docker containers, and future API workers. The database keeps two related concepts separate: `source_cache` stores reusable retrieval results, while `research_sessions` stores an auditable history of complete user research requests.

```
-- Reusable source-fetch cache
```

```

CREATE TABLE source_cache (
  id SERIAL PRIMARY KEY,
  source_type TEXT NOT NULL,
  canonical_query TEXT NOT NULL,
  data JSONB NOT NULL, -- list[Source]
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (source_type, canonical_query)
);

CREATE INDEX idx_source_cache_query
  ON source_cache (source_type, canonical_query);

-- Full research request history
CREATE TABLE research_sessions (
  session_id TEXT PRIMARY KEY,
  question TEXT NOT NULL,
  allowed_sources TEXT[] NOT NULL,
  retrieved_sources JSONB NOT NULL, -- list[Source]
  final_answer JSONB, -- AnswerWithCitations |
  NULL
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  execution_duration FLOAT NOT NULL DEFAULT 0.0
);

```

`source_type`, `canonical_query`, `question`, `allowed_sources`, `created_at`, and `execution_duration` are source-of-truth fields produced by our application. The JSONB payloads — `source_cache.data`, `research_sessions.retrieved_sources`, and `research_sessions.final_answer` — are derived from external source APIs and from the LLM synthesis result. The `UNIQUE` constraint on (`source_type`, `canonical_query`) supports safe upserts: repeated queries update the existing row instead of creating duplicate JSON files.

5.2 Caching policy

Required. What is cached, how long, where, and how is it invalidated. Report cache hit rate from a demo run.

- **What:** The merged `list[Source]` returned by `gather_all_sources` for a given (`query`, `sources`) pair. We also persist full `ResearchSession` rows for debugging and demo traceability. The LLM synthesis step is still not treated as the primary reusable cache value, so repeated runs can resynthesise from cached sources.
- **Key:** `source_type` + `canonical_query`, where the canonical query is `lower(question) + "|sources=" + sorted(enabled)`. Case-insensitive repeated questions hit the same row, but changing selected sources creates a different key.
- **TTL:** `CACHE_TTL_SECONDS` (default 14400s = 4 hours). TTL is checked at read time using the stored timestamp. The CLI flag `--no-cache` and the Streamlit cache toggle can bypass lookup for a single request.
- **Backend:** Database store selected through configuration (for example `CACHE_BACKEND=postgres`) using `DATABASE_URL`. Compared with the previous JSON cache, this gives atomic writes, easier query history inspection, and safer sharing between the CLI and UI.
- **Invalidation:** Time-based invalidation only. Expired rows are ignored on lookup and replaced on the next successful fetch using an upsert.
- **Cache hit rate (5-question demo, second run):** 100

Cached source lists can still become stale if a Wikipedia article changes or a new arXiv paper is published. The 4-hour TTL is a pragmatic trade-off between freshness and API politeness for

a research-assistant use case; a production system could add source-specific TTLs or scheduled cleanup for expired rows.

6. Testing

6.1 Strategy & coverage

Required. Report the coverage number. State how you mocked the AI module and HTTP layer. Confirm the provided smoke tests still pass.

Suite	Coverage
src/ (SE layer)	68 %
Provided ai/ smoke tests	passing

All tests are fully offline. The AI module is mocked via two fixtures in `confest.py`: `FakeLLM` (implements `LLMProvider`, returns a fixed cited string, records prompts in `.calls`) and `FakeWebSearch` (implements `WebSearchProvider`, returns a single canned `Source`). These are injected into `synthesize` and `fetch_web` via their optional `llm=` and `provider=` parameters, which means no monkeypatching of the `ai/` package internals is required.

HTTP calls in `test_concurrency.py` and `test_researcher.py` are mocked via `unittest.mock.AsyncMock` and `patch`, replacing `AsyncOrchestrator.gather_all_sources` and `CacheService.lookup/save` with controllable async fakes. `test_ai_smoke.py` exercises the `ai/` package internals (arXiv XML parser, citation index extractor, `Source` schema guards) through its public interface without any HTTP.

6.2 Notable tests

Required. Highlight at least three tests: one happy-path end-to-end, one concurrency test, one error-path test that surfaced a real bug.

- Happy-path end-to-end — `test_researcher.py::test_successful_research_and_cache`.** Mocks the orchestrator (returns one `Source`), the synthesiser (returns a fixed string), and the cache service. On the first call (cache miss) it asserts that `gather_all_sources` is called once and `CacheService.save` is called once. On the second call (cache hit) it asserts `gather_all_sources` is *not* called, proving the cache bypass works correctly.
- Concurrency / timeout — `test_concurrency.py::test_fetch_with_safety_timeout`.** Creates a coroutine that sleeps for 0.5s and an orchestrator with a 0.1s timeout. Asserts the return value is `[]` (not a `TimeoutError` propagation). This test caught a real bug: the original implementation raised `asyncio.TimeoutError` before the exception handler was in place, which would have killed the entire `gather_all_sources` call.
- Error-path CLI exit — `test_cli.py::test_handle_ask_exits_on_engine_error`.** Monkeypatches `ResearchAssistant` to raise `RuntimeError` and asserts `sys.exit(1)` is called. This test caught a missing `try/except` in the original `handle_ask` function, which would have printed a raw traceback to the user on any engine error.

Tests we did not write but wish we had: an integration test for the `FilesystemCacheStore` that verifies TTL expiry across two actual file reads (currently tested only at the `CacheService` level with mocked stores), and a test for the CLI's `--sources` filtering that traces the enabled tuple all the way through to the orchestrator.

7. Deployment & Reproducibility

7.1 Docker image

Required. Exact docker build and docker run commands. Image size. Python version. Single- or multi-stage build.

```
# Build
docker build -t research-assistant .

# Run a query (pass your .env file for API keys)
docker run --env-file .env research-assistant \
  python -m src.cli ask "What is quantum entanglement?"

# Run the offline demo (no keys needed)
docker run research-assistant python demo_ai.py --offline --limit 5
```

The image uses a **single-stage build** based on `python:3.13-slim` ([**TODO: fill in final image size, e.g. ~420 MB including all provider SDKs**]). We chose `slim` over `alpine` to avoid the `musl/glibc` incompatibility issues that `anthropic` and `numpy` exhibit on Alpine. A multi-stage build would reduce the final size by roughly 30 MB by excluding the compiler toolchain, but was not required for this project scope.

7.2 Configuration

Required. Every environment variable the app reads, and what happens if each is missing.

Variable	Default	If missing
LLM_PROVIDER	gemini	Uses default; no error
LLM_MODEL	gemini-2.5-flash	Uses default; no error
ANTHROPIC_API_KEY	—	ValidationError at startup if LLM_PROVIDER=anthropic
OPENAI_API_KEY	—	ValidationError at startup if LLM_PROVIDER=openai
GEMINI_API_KEY	—	ValidationError at startup if LLM_PROVIDER=gemini
WEB_SEARCH_PROVIDER	tavily	Uses default; no error
TAVILY_API_KEY	—	ProviderError at first web fetch if WEB_SEARCH_PROVIDER=tavily
SERPER_API_KEY	—	ProviderError at first web fetch if WEB_SEARCH_PROVIDER=serper
LOG_LEVEL	INFO	Uses INFO; no error
CACHE_BACKEND	postgres	Uses database-backed cache; validates DATABASE_URL
CACHE_DIR	—	Legacy JSON-cache option; not used by the database backend
CACHE_TTL_SECONDS	14400	Uses 24 h; no error
PER_SOURCE_TIMEOUT_SECONDS	10	Uses 10 s; no error
DATABASE_URL	—	Required when CACHE_BACKEND=postgres; ValidationError at startup if missing. Format: postgresql+asyncpg://user:pass@host/db

8. Limitations & Future Work

Required. 3–5 bullet points. Honest list of brittleness, productionisation gaps, and what you would change with another week.

- **Streamlit is a demo UI, not a production server.** The UI gives a clean browser workflow, but each button click still calls `asyncio.run()` inside the Streamlit script. A high-traffic version should move the research call behind FastAPI or another async service layer.
- **Database cache requires environment setup.** Moving away from JSON files makes concurrent access safer, but it adds an operational dependency: `DATABASE_URL` must be configured and the schema must be created before the CLI or UI can use the database backend.
- **LLM synthesis is not cached.** A repeated question returns cached sources but still pays an LLM call. Caching the `AnswerWithCitations` keyed on the source-set hash would make fully repeated questions free.
- **Keyword extraction adds latency on the hot path.** The `extract_keywords` LLM call happens *before* the parallel fetch, adding ~ 0.5 s even when the regex fallback would be good enough. Making the LLM extractor opt-in (e.g. `--smart-query` flag) would keep the default path fast.
- **No load testing.** We have not tested behaviour under sustained load (e.g. 100 concurrent requests). The Semaphore bound of 3 and the absence of a connection pool limit mean a burst of CLI invocations could saturate the Tavily free tier (1000 req/month) within minutes. A request budget counter persisted in the cache store would address this.

9. Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each contribution: which module, which assistant, whether the team reviewed and adapted it.

- **Claude (Anthropic)** assisted in drafting the initial structure of `src/concurrency/orchestrator.py`, specifically the `_fetch_with_retry` loop and back-off schedule. The team reviewed the output, identified that the original draft did not correctly propagate `asyncio.TimeoutError` through `asyncio.gather`, and rewrote the exception handling accordingly.
- **Claude** also generated the first draft of `tests/test_cli.py`. The team added the `test_handle_ask_exits_on_engine_error` case after the generated tests revealed the missing `try/except` in `handle_ask`.
- **No AI assistant** was used for `src/config.py`, `src/models.py`, or the caching layer; these were written by hand following the Pydantic documentation.

References

- [1] Anthropic API documentation. <https://docs.anthropic.com/>
- [2] Pydantic Settings documentation. https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [3] httpx async HTTP client. <https://www.python-httpx.org/>
- [4] Python `asyncio` documentation — Tasks and Coroutines. <https://docs.python.org/3/library/asyncio-task.html>

- [5] Tavily Search API. <https://docs.tavily.com/>
- [6] arXiv API User Manual. <https://info.arxiv.org/help/api/>
- [7] Streamlit documentation. <https://docs.streamlit.io/>
- [8] PostgreSQL documentation. <https://www.postgresql.org/docs/>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of those in the table.

```
git clone <repo> && cd <repo>
cp .env.example .env          # fill in LLM_PROVIDER, API keys, DATABASE_URL
pip install -r requirements.txt

# Browser UI
streamlit run src/ui.py

# CLI benchmark
# Clear cache / database rows, run 5 questions concurrently (default)
rm -rf .cache_storage
time python -m src.cli ask "What is photosynthesis?" --no-cache
time python -m src.cli ask "Explain transformer architecture" --no-cache
time python -m src.cli ask "What is quantum entanglement?" --no-cache

# Sequential baseline: temporarily set Semaphore(1) in orchestrator.py
# and re-run the same questions to measure sum-of-three fetch times.
```

End of report — thank you for reading.